Rim Defect Detection

Complete Notebook Guide

Understand every line of the rim-defect detection notebook,
even if you have never done deep learning before

Notebook: computer-vision-notebook_v4.ipynb

Updated: May 2026 | 28 Sections

Table of Contents

1. What Are We Building?

Imagine you work at a factory that makes car wheels (also called **rims**). After each rim is manufactured, it needs to be inspected for defects -- scratches, cracks, dents, or strange spots on the surface. Doing this by hand is slow, expensive, and people get tired and miss things.

We are building a **computer vision system** that looks at photos of rims and automatically decides: **"Is this rim defective or not?"**

The system uses **deep learning**, which is a type of artificial intelligence that learns from examples. We show it thousands of photos of rims (some good, some defective) and it figures out the patterns on its own.

The Two-Stage (Cascade) Approach

Think of this like airport security:

1. **Stage 1 (The Fast Scanner):** A quick, lightweight model looks at every single rim. If it's VERY confident the rim is defective, it flags it immediately. No need for the expensive scanner.
2. **Stage 2 (The Full Body Scan):** Only for rims that Stage 1 was unsure about. A powerful ensemble of 3 models looks very carefully at the rim surface and makes the final decision.

This two-stage approach saves time: Stage 1 handles the easy cases, Stage 2 focuses on the hard ones.

2. The Dataset

What do the images look like?

Each image is a photograph of a car rim, taken from a fixed camera angle. The images are:

- **1280 pixels wide × 1024 pixels tall** (about the size of a computer screen)
- **Colour images** (Red-Green-Blue channels, like any digital photo)
- The rim is somewhere in the middle of the photo

The problem: The rim is small

The actual rim only takes up about 40% of the image. The rest is background -- the floor, the wall, the camera mount. If we feed the whole image to the model, it will waste time looking at the background. So we **crop** the image to focus only on the rim.

Class imbalance (the big challenge)

- **82.4% of images are "GOOD"** (no defect) -- let us call these **negatives**
- **17.6% of images are "DEFECTIVE"** (has a scratch, crack, etc.) -- let us call these **positives**

This is like trying to learn what a "poisonous mushroom" looks like when 82 out of every 100 mushrooms are safe. The model will be lazy and just guess "safe" every time, getting 82% accuracy -- but missing every defective rim!

We fix this with two tricks:

1. **Oversampling:** We take extra copies of the defective images so the model sees an equal number of good and defective examples
2. **Weighted Loss:** We tell the model "missing a defect is 9 times worse than a false alarm" so it tries harder

Training / Validation Split

We split the data:

- **80% for training** -- the model learns from this
- **20% for validation** -- we keep this hidden and use it only to check how well the model is doing

We also use **5-fold cross-validation**: we split the data into 5 piles and train 5 separate models, each time using a different pile for validation. This gives us 5 models that we average together for better results.

3. Environment Setup & Imports

What happens in this cell?

Before we can do anything, we need to set up the computer's environment. Think of this like setting up a kitchen before cooking.

Critical: CUDA memory config

```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:64"
```

What is CUDA? CUDA is NVIDIA's technology that lets us run calculations on the graphics card (GPU) instead of the main processor (CPU). GPUs are much faster at deep learning.

Why this line? This tells PyTorch (our deep learning library) to keep its memory allocations small. On Kaggle GPUs (which have 15 GB of RAM), memory can get fragmented -- like a hard drive where files are scattered everywhere. This setting prevents crashes.

The imports

```
import torch              # The main deep learning library
import torch.nn as nn     # Neural network building blocks
import torchvision        # Pre-built models (backbones) and transforms
import cv2                # OpenCV -- image processing
import numpy as np        # Maths with arrays
import pandas as pd       # Reading CSV files (like Excel)
from PIL import Image     # Opening and saving images
from tqdm import tqdm     # Progress bars
```

Optional imports (with safety nets)

Some libraries are optional -- they make things better but the code still works without them:

- **albumentations** -- Better image augmentations (we transform images in smarter ways)
- **sklearn** -- For calculating AUC (a performance metric) and Platt scaling (calibrating probabilities)
- **torch._dynamo / torch.compile** -- Makes the model run faster on newer GPUs

Each optional import uses a try/except block -- if the library is not installed, the code sets a flag (like _ALBU_OK = False) and works around it.

ImageNet Normalisation Constants

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]
```

These are the average (mean) and spread (standard deviation) of pixel values across the millions of images in the ImageNet dataset. Our pretrained models were trained on ImageNet images, so we need to normalise our rim images to look the same.

Toddler explanation: Imagine you measure everyone's height in centimetres. But your friend measured the same people in inches. The numbers are different! ImageNet normalisation is like converting all measurements to the same unit so the model understands them.

4. Configuration (CLI Arguments)

What is a CLI argument?

CLI stands for **Command-Line Interface**. When you run a Python script, you can pass options like:

```
python notebook.py --batch 24 --lr 0.0001
```

These options change how the program behaves without editing the code.

The argument parser

```
import argparse
parser = argparse.ArgumentParser()
```

This creates a system that reads the command-line options and stores them in a variable called args. We can then access args.batch, args.lr, etc. throughout the code.

Key Arguments

File Paths

Argument	Default	What it does
`--dataset`	`"../input/dataset-cropped-26-v6"`	Folder containing cropped rim images
`--csv`	`"../input/dataset-cropped-26-v6/train.csv"`	CSV file with image IDs and labels
`--out`	`"assets/model_v6_fold_best.pt"`	Where to save the best checkpoint
`--resume`	`None`	Path to a checkpoint to resume training from

Training Schedule

Argument	Default	What it does
`--phase1-epochs`	5	Epochs to train only the "head" (new layers)
`--phase2-epochs`	8	Epochs to train head + last blocks
`--phase3-epochs`	27	Epochs to train the entire model
`--early-stop`	12	Stop training if no improvement for this many epochs
`--swa-start`	30	When to start Stochastic Weight Averaging
`--folds`	5	Number of cross-validation folds

What is an epoch? One epoch = one complete pass through the entire training dataset. If you have 10,000 images, one epoch means the model has seen all 10,000 images once.

Hyperparameters

Argument	Default	What it does
`--batch`	24	Number of images processed at once
`--lr`	5e-5	Learning rate (how big each step is)
`--img-size`	256	Size of input images (in pixels)
`--label-smooth`	0.02	Softens labels to prevent overconfidence
`--mixup-alpha`	0.1	Strength of MixUp augmentation
`--focal-gamma`	2.0	Focus of Focal Loss on hard examples
`--ema-decay`	0.9998	Smoothing factor for EMA
`--grad-clip`	0.5	Maximum gradient value (prevents explosions)

`--accum-steps`	4	Gradient accumulation steps
`--dropout-rate`	0.35	Dropout probability
`--val-split`	0.2	Fraction of data for validation

What is learning rate? Imagine you are trying to find the bottom of a valley in the dark. You take steps. If your steps are too big, you might step right over the bottom. If they are too small, it takes forever. Learning rate is the size of your step.

Flags

Argument	Default	What it does
`--bf16`	True	Use bfloat16 precision (faster on new GPUs)
`--channels-last`	True	Alternative memory layout (faster)
`--compile`	False	Use torch.compile (even faster, experimental)
`--oversample`	True	Duplicate minority class to balance data
`--pseudo-label`	False	Enable pseudo-labeling after training
`--cache-val`	False	Cache validation images in RAM

How this runs on Kaggle

On Kaggle, there is no command line (you are in a web browser). So the code does:

```
# If running in a notebook (no command line)
if len(sys.argv) == 1 or notebook_mode:
    args = parser.parse_args(args=[])
else:
    args = parser.parse_args()
```

When args=[] is passed, every argument uses its default value.

Special: Swin needs img_size = 256

Swin Transformer requires the image size to be divisible by its window size (which is 8). The code overrides any other img_size to 256:

```
if args.img_size != 256:
    print(f"[WARN] Swin needs 256. Overriding to 256.")
    args.img_size = 256
```

5. Stage 1: EfficientNet-B0 (The Fast Filter)

What is Stage 1?

Stage 1 is a SEPARATE, smaller model that runs BEFORE the main ensemble. Its job is:

- **Look at every test image**
- **If confident the rim is defective -> flag it immediately** (save the ensemble's time)
- **If unsure -> let the ensemble decide**

Why is it separate?

EfficientNet-B0 is tiny (only 5 million parameters) compared to the ensemble (50+ million parameters). It runs very fast. By filtering out easy cases, we only run the expensive ensemble on the hard cases, saving compute time.

Why full images?

Stage 1 was trained on **full uncropped images** (1280×1024 resized to 384×384), NOT on rim crops. This is important because:

- Stage 1 sees the entire image, including background
- It learns context: "if the whole rim area looks OK, probably fine"
- The ensemble sees only the cropped rim (256×256) -- zoomed in on the details

Training Stage 1

Stage 1 trains EfficientNet-B0 from scratch (well, from ImageNet pretrained weights) using:

- **Binary classification** (defective vs. good)
- **BCEWithLogitsLoss** with `pos_weight = 1.0` (the data is oversampled to 1:1 balance, so no extra weight needed)
- **AdamW optimizer** with learning rate 1e-4
- **Cosine annealing** learning rate schedule

The checkpoint is saved to STAGE1_WEIGHTS_OUT.

Important bug fix: Double-dip imbalance

The bug: The original code oversampled the minority class to create a 50/50 balanced dataset, BUT also used a pos_weight of ~9.0. This is like:

1. Making sure you have equal numbers of apples and oranges
2. Then punishing yourself 9 times harder if you miss an orange

The model panics and guesses "orange" on everything.

The fix: Set pos_weight = 1.0 when oversampling to 1:1. Balanced data = balanced weights.

6. Custom Augmentation Classes

What is data augmentation?

Augmentation means creating fake new training images by slightly modifying real ones. If you have 10,000 photos, augmentation can create 100,000+ different versions by rotating, flipping, adding noise, etc.

Why? Because the model learns to be **invariant** -- it learns that a scratch is still a scratch whether the image is flipped horizontally or not. This prevents the model from memorising specific images.

Our custom augmentations

These are specific to rim defect detection. They simulate real-world problems:

RadialBlur

```
class RadialBlur(A.ImageOnlyTransform):
```

Simulates **motion blur** from the rim spinning. When a rim rotates quickly during imaging, the camera might catch it mid-spin, creating blur lines radiating from the centre.

How it works: Applies a diagonal blur kernel (a small matrix that smears pixels) at a random angle.

SpecularHighlight

Simulates **reflections** from shiny metallic surfaces. Rims are shiny -- factory lighting creates bright spots (specular highlights) that can look like defects.

How it works: Adds a random elliptical bright spot somewhere on the image.

ScratchSimulation

Draws artificial **scratches** on the image -- thin, random lines at random angles. This teaches the model to recognise real scratches even when they look slightly different from the training examples.

SectorMask

Blacks out (fills with the average colour) a random **pie-slice** of the image. This simulates:

- Occlusion (something blocking part of the rim)
- Partial capture (the rim wasn't fully in frame)
- The model learning to detect defects even when part of the rim is hidden

RimSpecificAug

Picks one of three modes randomly:

1. **"vignette"** -- Darkens the edges of the image (like old photographs), simulating uneven lighting
2. **"ring"** -- Adds a faint circular ring, simulating a camera artefact
3. **"fixture"** -- Adds a thick bar on one side, simulating the mounting fixture that holds the rim

7. Image Transforms

Two pipelines: Training and Validation

`_make_train_transform(img_size, heavy=True)`

This is the **training** transform -- it includes random augmentations to make the model robust.

Light mode (heavy=False) -- Phases 1 & 2

Used when the backbone is partially frozen. Faster, simpler augmentations:

```
A.RandomRotate90()
A.HorizontalFlip()
A.VerticalFlip()
A.RandomBrightnessContrast()
A.GaussNoise()
A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
```

Heavy mode (heavy=True) -- Phase 3

Everything from light mode, PLUS:

```
A.ShiftScaleRotate()       # Shift, zoom, rotate
A.ElasticTransform()       # Stretch like rubber
A.HueSaturationValue()     # Change colours
A.CLAHE()                  # Improve local contrast
A.RandomGamma()            # Change brightness curve
A.ISONoise()               # Simulate camera sensor noise
A.MultiplicativeNoise()    # Random pixel noise
A.MotionBlur()             # Simulate camera movement
A.GaussianBlur()           # Soften the image
A.MedianBlur()             # Salt-and-pepper noise removal
A.CoarseDropout()          # Random rectangular erasures
# Our custom augs:
RadialBlur()
SpecularHighlight()
ScratchSimulation()
SectorMask()
RimSpecificAug()
```

Fallback

If albumentations is not installed, the code falls back to standard torchvision transforms (only basic flips and colour jitter).

`_make_val_transform(img_size)`

This is the **validation** transform -- NO randomness. Every validation run produces identical results, so we can trust the metrics:

```
A.Resize(img_size, img_size)
A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
A.ToTensorV2()
```

Note: RimDataset already resizes images internally, but having the Resize here ensures consistency between validation and test transforms.

8. Dataset Class (RimDataset)

What is a Dataset in PyTorch?

A Dataset is like a waiter at a restaurant. The kitchen (training loop) says "I need table 5's order" (get item at index 5). The waiter brings back: the image (the food) and the label (is it good or defective?).

RimDataset features

Constructor

```
class RimDataset(Dataset):
    def __init__(self, samples, transform, img_size=300, cache=False):
```

- **samples:** List of `(path_to_image, label)` tuples
- **transform:** The augmentation pipeline to apply
- **img_size:** Target size for the image
- **cache:** Whether to load all images into RAM at startup

Caching (important for speed)

When cache=True, the dataset loads ALL images into RAM when created. This is great for validation: instead of reading from disk (slow) every time, we read from RAM (fast).

Warning: If you cache >5000 images, the dataset prints a warning because it might exceed available RAM.

`_read_img(self, path, size)`

```
def _read_img(self, path, size):
    img = Image.open(path).convert("RGB")
    if img.size != (size, size):
        img = img.resize((size, size), Image.Resampling.BILINEAR)
    return np.array(img)
```

Opens the image with PIL, converts to RGB (in case it's grayscale), resizes to the target size, and returns as a NumPy array.

`__getitem__(self, idx)`

This is the "waiter" method:

1. Get the image path and label for index `idx`
2. If cached and available, use cached image; otherwise read from disk
3. Apply the transform (albumentations or torchvision)
4. Return `(tensor, label)` -- a PyTorch tensor and a float label

9. Data Loading

This section covers how we load, split, and serve data to the model.

`_load_samples(cropped_dir, csv_path)`

Reads the CSV file, finds the actual image files on disk, and creates a list of (path, label) tuples.

Important: The CSV stores just filenames (like "image_001.jpg"), not full paths. This function resolves each filename against the dataset directory.

`_split_samples(samples, val_frac, test_frac, seed)`

Splits samples into training, validation, and test sets.

Stratification: The split preserves the percentage of defective samples in each set. If the full dataset has 17.6% defective, both the training and validation sets also have ~17.6% defective.

`_oversample_minority(train_samples, target_ratio=1.0)`

Duplicates the minority class (defective examples) so both classes are equally represented.

How it works:

1. Count good and defective samples
2. If good=820, defective=180, we need to add 640 defective copies to make them equal
3. Randomly pick 640 defective images (with replacement -- some get picked multiple times)
4. Add them to the training set
5. Shuffle

Result: The model sees 50% good, 50% defective during training. It can't be lazy and guess "good" all the time.

`_make_sampler(train_samples)`

An ALTERNATIVE to oversampling. Instead of duplicating samples, we create a **WeightedRandomSampler** that:

- Gives each defective image a HIGHER probability of being selected
- Gives each good image a LOWER probability

This means every epoch, the model sees mostly defective images -- but they are STILL the original images, not copies.

The code uses oversampling by default (controlled by --oversample).

`_make_loaders(train, val, test, args, heavy_aug)`

Creates the three PyTorch DataLoaders:

Training DataLoader

- **Batch size:** `args.batch` (default 24)
- **Shuffle:** Yes (or uses sampler if not oversampling)
- **Drop last:** Yes (drops the last incomplete batch)
- **Workers:** Up to 4 parallel processes for loading images
- **Persistent workers:** Keeps workers alive between epochs (faster)

Validation DataLoader

- **Batch size:** `args.batch * 2` (48 default -- no gradient needed, so we can use larger batches)
- **Shuffle:** No (deterministic order)
- **Cache:** Controlled by `--cache-val` (default False)

Test DataLoader

- Same as validation, but smaller batch size (`args.batch`)

What are workers?

Loading images from disk is slow. Workers are separate processes that load images in parallel while the GPU is busy training. It is like having multiple waiters bringing food instead of just one.

10. Attention Modules

What is attention?

Imagine you are looking at a photo of a rim. Your eyes naturally focus on the scratch -- you do not look at every pixel equally. **Attention** is the same idea for neural networks: the model learns WHERE to look.

AttentionPool2d

Normal pooling averages ALL pixels equally:

```
Pooling output = (pixel_1 + pixel_2 + ... + pixel_N) / N
```

Attention pooling learns which pixels are important:

```
Attention output = weight_1 × pixel_1 + weight_2 × pixel_2 + ...
```

The weights are learned during training. The model learns to focus on the defect location (like a scratch) and ignore clean areas.

How it works:

1. A 1×1 convolution scores every spatial position
2. Softmax normalises these scores to sum to 1.0
3. The weighted sum of all positions becomes the output

CBAM -- Convolutional Block Attention Module

CBAM is like having two bouncers at a club:

Bouncer 1: Channel Attention

Each image has 3 colour channels (Red, Green, Blue). But inside the model, there are HUNDREDS of channels (each detecting different features -- edges, textures, patterns). Channel attention asks: "Which channels are most useful for detecting defects?"

How it works:

1. Average pool all positions -> get one value per channel
2. Max pool all positions -> get another value per channel
3. Feed both through a small neural network -> get importance scores per channel
4. Multiply each channel by its importance score

Bouncer 2: Spatial Attention

Spatial attention asks: "Which PIXELS are most important?" -- regardless of which channel they come from.

How it works:

1. Average pool across channels -> get one "average importance" map
2. Max pool across channels -> get one "max importance" map
3. Stack them -> 2-channel map
4. Run through a 7×7 convolution -> single-channel map
5. Multiply the original feature map by this spatial attention map

Result: The model focuses on the right CHANNELS at the right PIXELS.

11. EnsembleModelV6

The big idea

Instead of one model, we use THREE different models working together. They each look at the image differently, then discuss what they found (cross-attention) and vote on the final answer.

This is like having three doctors examine a patient:

- Doctor A (Swin Transformer) is great at seeing global patterns
- Doctor B (EfficientNet-B3) is great at efficiency and detail
- Doctor C (ConvNeXt-Small) is great at local texture analysis

The three backbones

1. Swin Transformer V2 Tiny

Type: Vision Transformer

Strength: Captures relationships between DISTANT parts of the image

Unlike CNNs that look at small local areas, Transformers look at the ENTIRE image at once. The "swin" in Swin stands for **Shifted Window** -- it divides the image into small windows and only attends within each window, then shifts the windows and attends again. This makes it efficient while still capturing global context.

Pretrained: On ImageNet (millions of everyday images). We fine-tune it for rim defects.

2. EfficientNet-B3

Type: Convolutional Neural Network (CNN)

Strength: Computationally efficient, good at fine details

EfficientNet was designed by Google using a technique called **Neural Architecture Search** -- an algorithm that automatically found the best way to scale width, depth, and resolution. B3 is the "medium" version.

3. ConvNeXt-Small

Type: Modern CNN

Strength: Combines the best of CNNs and Transformers

ConvNeXt is a modernised CNN that borrows ideas from Transformers (like GELU activation and LayerNorm) while keeping the efficiency of convolutions.

Architecture diagram (simplified)

```
Image
  ??? Swin-Tiny V2 -> CBAM -> AttentionPool -> Linear(320)
  ??? EfficientNet-B3 -> CBAM -> AttentionPool -> Linear(320)
  ??? ConvNeXt-Small -> CBAM -> AttentionPool -> Linear(320)
                          ?
                  [Cross-Attention Fusion]
                          ?
                  [Classifier Head]
                          ?
                  ????????????
              binary_head  severity_head
             (defective?)  (how bad?)
```

Cross-Attention Fusion

After each backbone produces a 320-dimensional feature vector, the three vectors are stacked as "tokens" and passed through a **Multi-Head Cross-Attention** layer (8 heads).

How this works:

1. Each backbone's output is a "token" (like a word in a sentence)
2. Cross-attention lets each token "look at" the other two tokens
3. If Swin sees a defect but EfficientNet is unsure, Swin's confidence influences EfficientNet

4. The result is a refined 960-dimensional vector (3 × 320)

Classification Head

The 960-dimensional vector goes through:

```
Linear(960 -> 512) -> GELU -> Dropout(0.35)
Linear(512 -> 256) -> GELU -> Dropout(0.35)
Linear(256 -> 128) -> GELU -> Dropout(0.35)
```

Then splits into two heads:

- **binary_head:** Linear(128 -> 1) with sigmoid -> probability of defect
- **severity_head:** Linear(128 -> 1) with sigmoid -> severity score (used only during training for regularisation)

SwinBridge -- The Shape Adapter

Swin Transformer outputs a 4D tensor of shape (Batch, Height, Width, Channels). But the rest of the model expects 3D (Batch, Channels, Height, Width).

SwinBridge permutes the dimensions:

```
# Input: (B, H, W, C) -- Swin output
x = x.permute(0, 3, 1, 2)
# Output: (B, C, H, W) -- what the rest expects
```

12. Loss Functions

What is a loss function?

The loss function is the **report card** for the model. After every batch of images:

1. The model makes predictions
2. The loss function compares predictions to the true labels
3. If predictions are wrong, the loss is HIGH
4. The model adjusts its weights to reduce the loss

FocalLossWithLogits

Standard Cross-Entropy: Treats all mistakes equally.

Focal Loss: Punishes HARD examples more, easy examples less.

Imagine a student studying for a test:

- Cross-Entropy = study every topic equally
- Focal Loss = focus more on the topics you keep getting wrong

Formula intuition: Focal = (1 - p)^gamma × CrossEntropy where p is the model's confidence. If the model is very confident and WRONG (p ? 1, wrong), gamma reduces the loss so the model focuses on other examples. If the model is unsure and RIGHT (p ? 0.5, correct), the loss stays high.

Asymmetric Loss (ASL)

Similar to Focal Loss, but treats positive and negative examples DIFFERENTLY:

- **gamma_pos = 1:** Light focus on hard positives (defects the model missed)
- **gamma_neg = 4:** Heavy focus on hard negatives (good rims the model thought were defective)

Why? Because false alarms (saying a good rim is defective) are more costly to fix than missed defects (a defective rim shipped to a customer).

CombinedLoss

Combines the two:

```
total = 0.35 × FocalLoss + 0.40 × ASL + 0.25 × severity_MSE
```

- **0.35 × FocalLoss:** Keep the standard focal weighting
- **0.40 × ASL:** Heavier weight on ASL (more aggressive on hard negatives)
- **0.25 × severity_MSE:** The severity head predicts a score (0 to 1). The MSE loss against the BINARY label acts as a regulariser -- if the model predicts high severity, the binary prediction should also be high.

13. EMA

What does EMA do?

EMA stands for **Exponential Moving Average**. It maintains a second copy of the model whose weights are the running average of ALL previous model states.

Toddler explanation: Imagine you are trying to draw a straight line but your hand is shaky. Every time you try, the line is slightly different. EMA takes the AVERAGE of all your attempts -- the average line is much straighter than any single attempt.

How it works

```
shadow_weights = decay × shadow_weights + (1 - decay) × current_weights
```

Every time the model updates its weights, the EMA copy also updates -- but it only moves 0.02% towards the new weights (with decay=0.9998). This means:

- The EMA model changes very slowly
- It averages out the noise in gradient updates
- The EMA model generalises better than the last checkpoint

Warmup decay

At the very start of training, the EMA starts with a lower decay and ramps up:

```
decay = min(decay, (1 + step) / (10 + step))
```

This lets the EMA catch up quickly at first, then stabilise.

Why use EMA?

Standard practice in machine learning competitions. The EMA model almost always performs better on validation data than the raw model at any point in training.

14. MixUp / CutMix

What problem do they solve?

Models can memorise training images. If you show the model the same scratch 100 times, it might learn "that exact scratch pattern" instead of learning "any scratch pattern". MixUp and CutMix prevent this by creating HYBRID images.

MixUp

Blends two images together like a double-exposure photograph:

```
mixed_image = 0.7 × image_A + 0.3 × image_B
mixed_label = 0.7 × label_A + 0.3 × label_B
```

The model sees an image that is 70% of one rim and 30% of another. The target label is also 70% / 30%. This forces the model to learn SOFT boundaries instead of hard decisions.

CutMix

Cuts a rectangle from image B and pastes it onto image A:

```
mixed_label = (area_of_rectangle / total_area) × label_B + (1 - ratio) × label_A
```

The model learns to detect defects even when part of the image is replaced with something else.

When are they used?

Only during Phase 2 and Phase 3 (when the backbone is unfrozen). They are disabled in Phase 1 (when only the head is training) to avoid confusing the newly initialised layers.

15. TTA

What is Test-Time Augmentation?

During inference (actual use), we run the image through the model MULTIPLE times with different transformations and average the results.

Toddler explanation: If you look at a photo upside down, you still recognise the face. TTA does the same for the model -- it shows the image in different orientations and asks "still defective?" If all orientations agree, the answer is more trustworthy.

How it works

```
def _tta_predict(model, imgs, device, n_views=4):
    predictions = []
    for each transformation (flip, rotate):
        transformed = apply_transform(imgs)
        pred = model(transformed)
        predictions.append(pred)
    return average(predictions)
```

8 D4 views

The "D4" group includes 8 transformations:

1. Original
2. Horizontal flip
3. Vertical flip
4. Both flips
5. Rotate 90°
6. Rotate 90° + flip
7. Rotate 180°
8. Rotate 270°

All 8 predictions are averaged to produce the final probability.

Why not always use TTA?

TTA is SLOW -- it runs the model 8 times per image instead of once. During training validation, TTA is optional (controlled by --tta-freq). During final inference, it is always recommended.

16. Validation & Threshold Sweep

`_validate(model, loader, device, ...)`

Runs the model on the entire validation set. Returns:

- **Accuracy:** What fraction of predictions were correct
- **F1 Score:** The harmonic mean of precision and recall (our main metric)
- **Precision:** Of all rims we called defective, how many were actually defective?
- **Recall:** Of all actually defective rims, how many did we catch?
- **AUC:** Area Under the ROC Curve -- a measure of ranking quality
- **Raw probabilities and labels:** Used later for threshold sweep

`_sweep_threshold(model, loader, device, ...)`

Why not use 0.5?

Default threshold (0.5) assumes balanced classes. With 82% good / 18% defective, the optimal threshold is usually NOT 0.5 -- it is often lower (0.3-0.4) to catch more defects.

How it works

Instead of looping through thresholds (slow), the code uses GPU broadcasting:

1. Get probabilities for all validation images
2. Create a tensor of thresholds: [0.05, 0.06, ..., 0.95]
3. In ONE operation, compare all probabilities against all thresholds
4. Calculate F1 for every threshold simultaneously
5. Return the threshold that gives the best F1

This is much faster than a Python loop.

17. BN Recalibration

What is Batch Normalisation?

Batch Normalisation (BN) is a technique that normalises the outputs of each layer so they have mean 0 and standard deviation 1. This stabilises training.

BN keeps **running statistics** -- it tracks the average mean and variance of every layer's outputs during training. At inference, it uses these running statistics instead of batch statistics.

Why recalibrate?

When we use the EMA model (which is an average of weights), the BN running statistics are STALE -- they were collected from the raw model, not the EMA model. If we use the EMA model with stale statistics, its outputs might be wrong.

`_recalibrate_bn(model, loader, device, n_batches=25)`

1. Put the model in training mode (so BN statistics update)
2. Run 25 batches through the model WITHOUT calculating gradients
3. The BN statistics now reflect the EMA model's behaviour
4. Switch back to eval mode

18. Platt Scaling

What is calibration?

Neural networks are often **overconfident**. A model might say "99% confident this rim is defective" when it is actually only 70% sure.

Platt Scaling fixes this by fitting a logistic regression model on the validation set:

```
# Input: raw model logit
# Output: calibrated probability
calibrated_prob = sigmoid(A × raw_logit + B)
```

Where A and B are learned to make the probabilities match reality.

When the model says 90%

After Platt scaling, if the model says 90%, it means: "Of all rims I assigned 90% probability to, about 90% were actually defective." This is called **calibration**.

How it is saved

The fitted calibrator (just two numbers: A and B) is saved to calibrator_fold_{n}.pkl and can be loaded at inference time.

19. Three-Phase Training

Why three phases?

When we start with a pretrained model (trained on ImageNet -- everyday objects like dogs, cats, cars), it doesn't know anything about rim defects. If we unfreeze ALL layers immediately, the model will destroy its pretrained knowledge before learning anything about rims.

Progressive unfreezing prevents this:

Phase 1: Head Warm-Up (5 epochs)

- **Frozen:** All backbone layers
- **Training:** Only the classification head, cross-attention, and fusion gate
- **Learning rate:** Full base_lr (5e-5)
- **Augmentations:** Light

Analogy: You hired an expert chef (pretrained backbone). You teach them how to use YOUR kitchen (head layers) before retraining them on YOUR recipes.

Phase 2: Head + Last Blocks (8 epochs)

- **Frozen:** Backbone stems (early layers)
- **Training:** Head + last few blocks of each backbone
- **Learning rate:** Head = base_lr, last blocks = 0.1 × base_lr
- **Augmentations:** Light

Analogy: The chef now learns to adjust their knife skills (last blocks) while keeping their core techniques (stem) unchanged.

Phase 3: Full Fine-Tuning (27 epochs)

- **Frozen:** Nothing
- **Training:** Everything
- **Learning rate:** Backbone = 0.02-0.04 × base_lr, Head = base_lr
- **Augmentations:** Heavy (MixUp/CutMix, all custom augs)

Analogy: The chef now retrains completely, but they still keep their fundamental knowledge (lower learning rate for backbone).

Phase transition code

```
def _setup_phase(model, phase, args, n_epochs):
    if phase == 1:
        # Freeze everything except head
        for name, param in model.named_parameters():
            if 'head' in name or 'fusion' in name or 'cross_attn' in name:
                param.requires_grad = True
            else:
                param.requires_grad = False
    elif phase == 2:
        # Unfreeze head + last blocks of each backbone
        ...
    elif phase == 3:
        # Unfreeze everything
        for param in model.parameters():
            param.requires_grad = True
```

20. AMP

What is Mixed Precision?

Normally, neural networks use **32-bit floating point numbers** (float32). AMP (Automatic Mixed Precision) uses **16-bit** for most calculations and 32-bit only where needed.

Toddler explanation: Float32 writes numbers with 7 decimal places of precision. Float16 writes numbers with 3 decimal places. Most of the time, 3 decimal places is enough -- and it is TWICE as fast and uses half the memory!

When to use what

The code automatically selects the best precision:

```
def _setup_amp(device, want_bf16):
    if device.type != "cuda":
        return False, torch.float32, None   # CPU = no AMP

    capability = torch.cuda.get_device_capability()

    if capability >= (8, 0):  # Ampere+ GPUs
        return True, torch.bfloat16, None   # bfloat16 (no scaler needed)

    elif capability >= (7, 0):  # Volta/Turing (V100, T4)
        return True, torch.float16, GradScaler()  # float16 + loss scaler
```

- **Ampere+ (A100, RTX 3090, L4):** bfloat16 -- stable, no loss scaler needed
- **Volta/Turing (V100, T4):** float16 needs a GradScaler to prevent underflow
- **CPU:** No AMP

What is a GradScaler?

When using float16, very small gradient values can underflow to zero (they are too small to represent in 16 bits). A GradScaler MULTIPLIES the loss by a scale factor (e.g., 2^16) before backpropagation, then divides the gradients by the same factor afterwards. This keeps small gradients from vanishing.

21. Training Loop

This is the heart of the notebook. Everything else is preparation for this.

High-level flow

```
for each fold in 5 folds:
    prepare data for this fold
    build model

    for phase in [1, 2, 3]:
        setup optimizer (phase-appropriate LRs)

        for epoch in range(n_epochs):
            train loop:
                for each batch:
                    apply MixUp/CutMix (phases 2-3)
                    forward pass (AMP autocast)
                    calculate loss
                    backward pass
                    gradient accumulation
                    optimizer step
                    EMA update

            validation (every N epochs):
                calibrate BN
                run validation
                sweep threshold
                if F1 improved:
                    save checkpoint
                else:
                    early stopping counter++

    finalise fold (SWA, calibration)
    save fold results
```

Detailed breakdown

1. Seed Setup

```
random.seed(args.seed + fold)
np.random.seed(args.seed + fold)
torch.manual_seed(args.seed + fold)
torch.cuda.manual_seed_all(args.seed + fold)
```

Different seed for each fold ensures each fold sees differently shuffled data.

2. Data Preparation

```
(train_loader, val_loader, _, _) = _make_loaders(
    train_samples, val_samples, [], args, heavy_aug=False)
```

3. Model Construction

```
model = build_model().to(device)
if args.channels_last and device.type == "cuda":
    model = model.to(memory_format=torch.channels_last)
```

Channels-last is a memory layout optimisation. Normally PyTorch stores images as (batch, channel, height, width) -- all of the same channel together. Channels-last stores as (batch, height, width, channel) -- all of the same pixel together. This is faster on NVIDIA GPUs.

4. Resume from Checkpoint

If --resume is provided, the code loads:

- Model weights (state_dict)
- Optimizer state (only if checkpoint phase matches current phase)
- Epoch counter
- Best F1 score

Bug fix: The optimizer is only restored when the checkpoint's phase matches the current training phase. Restoring a Phase 2 optimizer during Phase 1 would cause a crash.

5. Loss Setup

```
focal_fn = FocalLossWithLogits(gamma=args.focal_gamma, pos_weight=pos_weight, eps=args.label_smooth)
asl_fn = AsymmetricLoss(gamma_neg=4, gamma_pos=1, clip=0.05)
criterion = CombinedLoss(focal_fn, asl_fn, severity_weight=0.25, asl_weight=0.40)
```

6. Phase Loop

```
for phase, n_epochs in phases:
    optimizer = _setup_phase(model, phase, args, n_epochs)

    for epoch in range(start_epoch, n_epochs + 1):
```

When transitioning to Phase 3, the data loader is rebuilt with heavy augmentation:

```
if phase == 3:
    (train_loader, _, _, _) = _make_loaders(
        train_samples, val_samples, [], args, heavy_aug=True)
```

7. Training Batch Loop

```
for batch_idx, (imgs, lbls) in enumerate(train_loader):
    # MixUp/CutMix (phase 2+)
    if phase >= 2 and args.mixup_alpha > 0:
        imgs, lbls = _mixup_cutmix(imgs, lbls, alpha=args.mixup_alpha)

    # Forward pass with AMP
    with torch.amp.autocast(device_type=device.type, dtype=amp_dtype, enabled=use_amp):
        bin_logits, sev_logits = model(imgs)
        loss = criterion(bin_logits, sev_logits, lbls) / args.accum_steps

    # Backward pass
    if scaler:
        scaler.scale(loss).backward()
    else:
        loss.backward()

    # Gradient accumulation
    if (batch_idx + 1) % args.accum_steps == 0:
        if scaler:
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
            scaler.step(optimizer)
            scaler.update()
        else:
            torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
            optimizer.step()

        optimizer.zero_grad()

        if ema:
            ema.update(model)
```

8. Validation

```
if epoch % args.val_freq == 0:
    _recalibrate_bn(ema_model, train_loader, device)

    val_acc, val_f1, val_prec, val_rec, val_auc = _validate(
        ema_model, val_loader, device, threshold=0.5)

    best_thr, best_f1_val = _sweep_threshold(
        ema_model, val_loader, device)

    if best_f1_val > best_f1:
        best_f1 = best_f1_val
        # Save checkpoint
```

9. Early Stopping

If validation F1 doesn't improve for args.early_stop (12) consecutive epochs, training stops.

10. SWA Finalisation

After all epochs, if SWA was enabled:

```
if swa_model is not None:
    _recalibrate_bn(swa_model, train_loader, device)
    swa_f1 = _validate(swa_model, val_loader, device)
    if swa_f1 > best_f1:
        best_f1 = swa_f1
```

11. Platt Calibration

```
cal = _calibrate_model(ema_model, val_loader, device)
```

12. Cross-Validation Results

```
print("  CROSS-VALIDATION RESULTS")
for fold, f1, thr in fold_results:
    print(f"  Fold {fold+1}: F1={f1:.4f}  threshold={thr:.2f}")
print(f"  Mean F1: {mean_f1:.4f}")

# Save best fold
best_fold, best_f1, best_thr = max(fold_results, key=lambda x: x[1])
shutil.copy(best_ckpt, out_path)
print(f"Best fold {best_fold+1} saved to {out_path}")
```

22. Cascade Inference Pipeline

The two stages (detailed)

Stage 1: EfficientNet-B0 (Full Image)

1. Load the full 1280×1024 test image
2. Resize to 384×384
3. Apply standard ImageNet normalisation
4. Run through EfficientNet-B0
5. If probability > STAGE1_THRESHOLD -> **predicted DEFECTIVE, STOP** (no need for ensemble)
6. If uncertain -> pass to Stage 2

Stage 2: 5-Fold Ensemble (Cropped Rim)

1. Extract the rim crop from the image (256×256)
2. Apply validation transform (resize + normalise)
3. Run through all 5 fold models (each gets a vote)
4. Average the 5 probabilities
5. If average > mean_threshold -> DEFECTIVE, else GOOD

`cascade_predict(full_img_tensor, crop_tensor)`

```
def cascade_predict(full_img_tensor, crop_tensor):
    if stage1_model is not None:
        s1_logits = stage1_model(full_img_tensor.unsqueeze(0).to(device))
        s1_probs = torch.sigmoid(s1_logits).squeeze()
        if s1_probs >= STAGE1_THRESHOLD:
            return torch.tensor([1.0], device=device)  # Defective, skip ensemble
    return predict_ensemble(crop_tensor.unsqueeze(0))  # Unsure, run ensemble
```

`extract_rim_crop` -- The rim cropper

```
def extract_rim_crop(img_rgb, out_size=256, margin=0.05):
    h, w = img_rgb.shape[:2]
    # Relative coordinates (reference: 1280x1024)
    rel_cx = (364 + 526/2) / 1280.0
    rel_cy = (247 + 526/2) / 1024.0
    rel_side = 526.0 / min(1280.0, 1024.0)

    cx = max(0, int(w * rel_cx))
    cy = max(0, int(h * rel_cy))
    side = max(out_size, int(min(w, h) * rel_side * (1 + margin)))
    half = side // 2

    x1 = max(0, cx - half)
    y1 = max(0, cy - half)
    x2 = min(w, cx + half)
    y2 = min(h, cy + half)

    crop = img_rgb[y1:y2, x1:x2]
    if crop.size == 0:
        crop = img_rgb[h//4:3*h//4, w//4:3*w//4]  # fallback

    return cv2.resize(crop, (out_size, out_size))
```

Why relative coordinates? Originally the coordinates were hardcoded for 1280×1024 images. If test images have different dimensions, the crop would be wrong. Relative coordinates adapt to any image size.

Important: BGR -> RGB conversion

OpenCV reads images as BGR (Blue-Green-Red) by default. But the model was trained on RGB (Red-Green-Blue). So we must

convert:

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

If you forget this: The model sees blue as red and red as blue -> garbage predictions!

23. Pseudo-Labeling

What is pseudo-labeling?

After training the 5-fold ensemble, we have a powerful model. We can use this model to **guess** labels for the test images (which have no labels). Then we retrain a master model using both the original data AND our guessed labels.

Toddler explanation: The teacher (ensemble) grades some ungraded homework (test images). The student (master model) learns from both the teacher-graded homework AND the officially graded homework.

The process

Step 1: Ensemble inference on test images

All 5 fold models run inference on every test image. Each test image gets 5 probability scores. We average them.

Step 2: Select confident predictions

```
high_conf = all_probs >= 0.95    # Very likely defective
low_conf  = all_probs <= 0.05    # Very likely good
selected  = high_conf | low_conf
```

We only keep predictions where ALL 5 models agree (>0.95 OR <0.05). This ensures we only add high-quality pseudo-labels.

Step 3: Split ORIGINAL data (critical bug fix)

Original (wrong) approach: Mix pseudo-labels with original data, THEN split into train/val. This puts pseudo-labeled test images in the validation set! The model would be evaluated on its own guesses -- leading to falsely high F1 scores (~0.99).

Fixed approach:

1. Split ORIGINAL data first (val is pure ground-truth)

```
orig_labels = [l for _, l in all_samples]
train_idx, val_idx = train_test_split(
range(len(all_samples)), test_size=args.val_split,
stratify=orig_labels, random_state=args.seed + 999)
final_train = [all_samples[i] for i in train_idx]
final_val   = [all_samples[i] for i in val_idx]
```

2. Append pseudo-labels ONLY to training set

final_train.extend(pseudo_samples)

rng.shuffle(final_train)

```
### Step 4: Retrain master model
```

master_f1, master_thr = train_one_fold(

fold=999, train_samples=final_train, val_samples=final_val, ...)

```
The master model is trained with `fold=999` (special fold number for pseudo-label retraining). It is saved as `model_v6_master.pt`.

---
```

24. Evaluation & Metrics Report

How to evaluate a trained model

The run_evaluation() function in cell 42 provides a comprehensive evaluation:

1. Auto-detect best fold

```
assets_dir = Path(args.out).parent
fold_csvs = sorted(assets_dir.glob("val_split_fold_*.csv"))
if not fold_csvs:
    print("[WARN] No fold validation CSVs found. Skipping evaluation.")
    return
COMPARE_VAL_CSV = str(fold_csvs[-1])
```

2. Load validation data

Creates a RimDataset from the validation split CSV and loads the saved checkpoint.

3. Inference with TTA

```
if COMPARE_TTA:
    probs = _tta_predict(model, imgs, device, n_views=8).cpu().numpy()
else:
    logits, _ = model(imgs)
    probs = torch.sigmoid(logits).cpu().numpy()
```

4. Threshold sweep

Tests all thresholds from 0.05 to 0.95 and prints F1, Precision, Recall for each.

5. Final report

```
print(f"  Accuracy: {m['acc']:.2%} | F1: {m['f1']:.4f} | Prec: {m['prec']:.2%} | Rec: {m['recall']:.2%}")
print(f"  TP: {m['tp']} (Correct Faulty)   |  TN: {m['tn']} (Correct Good)")
print(f"  FP: {m['fp']} (False Alarms)     |  FN: {m['fn']} (Missed Defects)")
```

6. Misclassified images

Lists the worst misclassifications -- images where the model was most confidently WRONG. Useful for debugging.

25. Known Bugs Fixed

This notebook has undergone extensive bug fixing. Here is a summary of every bug found and fixed:

#	Bug	Impact	Fix
1	`import argparseparser` syntax error	**Crash**	Split into `import argparse` / `parser = argparse.ArgumentP
2	SwinBridge assumed sequence format	**Crash**	Changed to `x.permute(0, 3, 1, 2)` for 4D Swin output
3	Missing CBAM, AttentionPool2d, `_get_	**Crash**	Restored from backup notebook
4	BGR->RGB not converted (OpenCV BC	**Wrong predictions**	Added `cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)`
5	Inference used training transform (rand	**Non-deterministic**	Changed to `_make_val_transform()`
6	Fold detection at module level (Indenta	**Crash**	Moved inside function at 4-space indent
7	`_make_loaders` called transform witho	**Missing resize**	Added `args.img_size` argument
8	Stage 1 doubled imbalance (oversampl	**All predictions defective**	Set `pos_weight = 1.0`
9	Pseudo-label split included pseudo-labe	**Falsely high F1**	Split original data first, append pseudo to train only
10	`"optimizer" in locals()` instead of `"opti	**No optimizer restore**	Fixed condition
11	TTA block 20-space indent (should be 1	**IndentationError**	Fixed indent level
12	Training depended on undefined global	**Crash**	Passed as parameters
13	Duplicate `--seed` in argparse	**Confusing**	Removed duplicate
14	Hardcoded `val_split_fold_1.csv`	**Wrong file**	Dynamic fold detection + best-fold matching
15	Optimizer restored unconditionally on re	**Phase mismatch**	Only restore when checkpoint phase matches
16	Hardcoded crop coordinates (1280×102	**Wrong crop on other sizes**	Relative coordinates
17	Batch cap at 32	**Underutilised GPU**	Removed cap
18	`raise FileNotFoundError` when no fold	**Ugly crash**	Graceful print + return
19	Duplicate `STAGE1_IMG_SIZE`	**Confusing**	Commented duplicate
20	No RAM warning for large cache	**OOM crash**	Warning if >5000 images
21	`ckpt` variable scope fragile	**Potential NameError**	Initialize `ckpt = {}` unconditionally
22	Warmup LR negative on tiny datasets	**Wrong LR**	Guard `warmup_steps > 0`
23	Dead `SwinSequenceAdapter` class	**Dead code**	Removed
24	Duplicate `if __name__` entries	**Double execution**	Removed

26. OHEM -- Online Hard Example Mining

What problem does it solve?

Your dataset has 35,342 images. Most of them are **easy** -- the model quickly learns to classify them correctly. If we train on all images equally, the model wastes 90% of its learning energy on examples it already knows perfectly.

OHEM (Online Hard Example Mining) solves this by dynamically filtering each mini-batch during training to keep only the images the model is struggling with most.

The Toddler Explanation

Imagine you are studying for a vocabulary test. You have 100 words. You already know 70 of them perfectly. Instead of going through all 100 every time, a smart teacher would only quiz you on the 30 words you keep getting wrong. That is OHEM.

How it works in the code

The OHEMLoss class wraps the existing CombinedLoss:

```
criterion = OHEMLoss(CombinedLoss(
    FocalLossWithLogits(...),
    AsymmetricLoss(),
), keep_frac=0.7)
```

Every training step:

1. A batch of 24 images is passed through the model
2. The loss is computed **individually** for each of the 24 images
3. The 24 losses are sorted from highest (hardest) to lowest (easiest)
4. The **bottom 30% (easiest images) are thrown away**
5. The gradient update only uses the top 70% hardest images

```
class OHEMLoss(nn.Module):
    def forward(self, bin_logits, sev_logits, targets):
        per_sample = [self.base(bin_logits[i:i+1], ...) for i in range(B)]
        per_sample = torch.stack(per_sample)        # (B,) individual losses
        k = max(1, int(B * self.keep))              # keep top 70%
        hard, _ = per_sample.topk(k, largest=True) # select hardest k
        return hard.mean()                          # backprop only on these
```

VRAM and time impact

- **VRAM:** Zero extra. You are doing the same forward pass, just ignoring part of the backward pass.
- **Time:** Adds approximately 2-3% per epoch (the per-sample loop). Virtually free.
- **Expected gain:** Forces the model to concentrate on the subtle micro-scratches and borderline cases instead of the obvious defects it already handles.

What `keep_frac=0.7` means

`keep_frac`	Effect
`1.0`	Same as normal loss (no OHEM, all samples kept)
`0.7`	Keep the 70% hardest -- recommended starting point
`0.5`	Very aggressive -- only the worst half drives learning
`< 0.4`	Too aggressive -- model may become unstable

27. Grad-CAM Dynamic Cropping

What problem does it solve?

The current pipeline crops the rim to a fixed 384×384 square from the centre of the image. If a scratch is very thin (e.g., 3 pixels wide on the original image), it becomes 3 pixels wide in a 384×384 crop. When EnsembleV6 looks at this, the scratch might be invisible among all the other pixels.

Grad-CAM Dynamic Cropping gives the ensemble a **magnifying glass** focused exactly on the suspicious area that Stage 1 (B0) already spotted.

What is Grad-CAM?

Grad-CAM (**Gradient-weighted Class Activation Mapping**) is a technique that produces a heatmap showing **which pixels of the image most influenced the model's decision**.

Toddler explanation: After the model says "I think this rim is defective", Grad-CAM asks it "*why?* Where did you see the defect?" The answer is a coloured heat-map -- bright red over the crack, dark blue over the clean areas.

How it works step by step

Step 1 -- Forward pass + gradient

```
logit = b0_model(image)
logit.backward()  # compute gradients w.r.t. last conv layer
```

Step 2 -- Weight the feature maps

```
weights = gradients.mean(dim=(H, W))      # average gradient per channel
cam = (weights × feature_maps).sum()     # weighted sum of feature maps
cam = relu(cam)                           # keep only positive contributions
```

Step 3 -- Resize heatmap to image size

The heatmap starts small (e.g., 7×7 from the last conv layer). It is resized to the full image (e.g., 384×384) using bilinear interpolation.

Step 4 -- Find the hotspot & crop

```
cy, cx = argmax(cam_full)              # pixel with highest activation
half   = image_size * 0.65             # crop size (65% of the image)
patch  = image[cy-half:cy+half, cx-half:cx+half]
patch  = resize(patch, (384, 384))     # resize patch to model input size
```

The confidence gate

Grad-CAM is only applied when B0 **thinks there is a defect** (prob ? 0.4). If B0 gives a very low probability (clean rim), we fall back to the standard rim crop -- no point zooming into a hotspot that doesn't exist.

```
if b0_prob >= 0.4:   # B0 suspects a defect
    crop = _gradcam_crop(img_rgb, b0_model, ...)  # zoom into hotspot
else:                # B0 thinks it's clean
    crop = extract_rim_crop(img_rgb, img_size)    # standard crop
```

Architecture flow (updated)

```
Full Image
    |
    v
[Stage 1: B0]  ??? prob < 0.4 ????????????????????????????????????
    |                                                              |
    | prob >= 0.4                                                  |
    v                                                              |
[Grad-CAM Heatmap]                                                |
    |                                                               |
    v                                                               |
[Dynamic Crop: zoom into hotspot]     [Standard Rim Crop] <???????
    |                                          |
    ??????????????????????????????????????????
                    |
                    v
         [EnsembleV6: Swin + EffNet + ConvNeXt]
                    |
                    v
           [Final Prediction]
```

VRAM and time impact

- **VRAM:** B0 is kept in memory during Stage 2 instead of being deleted. This costs approximately **15 MB** extra (B0 is tiny).
- **Time:** Grad-CAM requires one forward+backward pass per image at inference. With `torch.enable_grad()` scoped only to the Grad-CAM call, this adds roughly **10-15% to inference time** only (not training).
- **Expected gain:** On borderline images where the defect occupies <5% of the crop, zooming in can be the difference between a correct and incorrect prediction.

Key implementation detail -- `torch.enable_grad()`

Normally, the entire inference loop runs inside torch.no_grad() to save memory. Grad-CAM **requires gradients** (that is how the heatmap is computed). We use torch.enable_grad() scoped only to the Grad-CAM call:

```
with torch.no_grad():
    for batch in test_loader:
        ...
        with torch.enable_grad():      # only here
            crop = _gradcam_crop(...)  # needs grad
        ...
        logits = ensemble(crop)        # back to no_grad
```

This keeps memory usage minimal while still computing the heatmap correctly.

28. Glossary -- Terms Explained Like You Are 5

A

Accuracy -- How many answers the model got right out of all answers. If 90 out of 100 are correct, accuracy is 90%.

AdamW -- A fancy way for the model to decide HOW MUCH to change its weights each step. Like a smart hiker who adjusts step size based on the terrain.

AMP (Automatic Mixed Precision) -- Using smaller numbers (16-bit instead of 32-bit) to make calculations twice as fast. Like counting in "dozens" instead of counting each item one by one.

Argparse -- A tool that reads settings from the command line. Instead of editing the code to change batch size, you type --batch 24 when you run the program.

Asymmetric Loss (ASL) -- A loss function that penalises false negatives (missed defects) more than false positives (false alarms).

Attention -- A mechanism that lets the model focus on important parts of an image instead of all pixels equally.

AUC (Area Under Curve) -- A metric for binary classifiers. AUC = 1.0 is perfect; AUC = 0.5 is random guessing.

AUC (Area Under the ROC Curve) -- A number from 0 to 1 that measures how good the model is at ranking. 1.0 = perfect, 0.5 = random guessing.

Augmentation -- Creating fake new training images by slightly changing real ones. Like turning a photo upside down to create a "new" photo.

B

Backbone -- The main part of a neural network that extracts features from images. Usually pretrained on millions of everyday photos.

Batch -- A group of images processed at the same time. Instead of looking at one image at a time, the model looks at 24 images together.

Batch Normalisation (BN) -- A technique that keeps numbers in a healthy range as they flow through the network. Like making sure water pressure stays constant in pipes.

BatchNorm (BN) -- A layer that normalises activations to prevent them from becoming too large or too small during training.

BCEWithLogitsLoss -- A loss function for binary classification (good vs. defective). Combines a sigmoid activation and binary cross-entropy in one stable operation.

Bfloat16 -- A 16-bit number format used in newer GPUs. More stable than float16 because it keeps more exponent bits (for very large/small values).

Bfloat16 (bf16) -- A 16-bit floating point format that is more numerically stable than float16. Requires a modern GPU (e.g., A100).

Binary classification -- A task with exactly two possible outputs: defective (1) or clean (0).

C

Calibration -- Making the model's confidence match reality. If the model says 90%, it should be right 90% of the time.

CBAM -- Convolutional Block Attention Module. A "where to look" mechanism that helps the model focus on important channels and pixels.

Channels -- Each image has 3 colour channels (Red, Green, Blue). Inside the model, there can be hundreds of channels, each detecting different features (edges, textures, etc.).

Checkpoint -- A saved snapshot of the model's weights at a specific point during training.

Class imbalance -- When one class has many more examples than the other. Here: 82% defective vs 18% clean.

CLI (Command-Line Interface) -- Running a program by typing commands instead of clicking buttons.

CNN (Convolutional Neural Network) -- A type of neural network that uses small filters (convolutions) to scan images. Like moving a magnifying glass over a photo.

ConvNeXt -- A modern CNN that borrows ideas from Transformers while keeping convolutional efficiency.

Cross-Attention -- A mechanism where different parts of a model "talk to each other." Each backbone tells the others what it found, and they refine each other's conclusions.

Cross-entropy loss -- The standard loss function for classification. Measures the difference between predicted probabilities and true labels.

Cross-Validation -- Splitting data into several piles and training separate models, each using a different pile for testing. Like taking 5 different tests instead of 1.

CUDA -- NVIDIA's technology for running calculations on the GPU. Makes deep learning much faster.

CutMix -- A data augmentation technique that cuts a rectangle from one image and pastes it onto another, mixing their labels proportionally.

D

DataLoader -- PyTorch's tool for feeding data to the model in batches, with optional shuffling and parallel loading.

Dataset -- In PyTorch, a class that knows WHERE to find each image and what its label is.

Defect -- A flaw in the rim (scratch, crack, dent, spot).

Dropout -- Randomly turning off some neurons during training. Prevents memorisation. Like studying without some of your notes to force deeper understanding.

Dynamic Cropping (Grad-CAM) -- Using the B0 heatmap to crop the image around the suspected defect zone before passing it to the ensemble.

E

Early stopping -- Stopping training when performance stops improving to avoid overfitting.

EfficientNet -- A family of CNN models designed by Google to be efficient (good accuracy for their size). B0 is tiny, B3 is medium, B7 is huge.

EMA (Exponential Moving Average) -- Keeping a slow-moving average of the model's weights. The average model is usually better than the latest model.

Ensemble -- Combining multiple models and averaging their predictions. Like asking 5 doctors instead of 1.

Epoch -- One complete pass through the entire training dataset. If you have 10,000 photos and batch size 24, one epoch = 417 batches.

F

F1 Score -- The harmonic mean (fancy average) of precision and recall. Our main metric. Ranges from 0 (terrible) to 1 (perfect).

F1-Score -- The harmonic mean of precision and recall. F1 = 1.0 is perfect; F1 = 0.0 is worst.

False Negative (FN) -- A defective rim classified as clean. Very costly in a factory setting.

False Positive (FP) -- A clean rim classified as defective. Causes unnecessary rework but is less dangerous.

Feature map -- The internal representation of an image at a specific layer of the network. Like an X-ray highlighting certain patterns.

Fine-Tuning -- Taking a pretrained model and continuing to train it on your specific data.

Float16 -- 16-bit floating point numbers. Half the precision of float32 but twice as fast. Can underflow on very small values.

Float32 -- Standard 32-bit floating point numbers. Used by default in neural networks.

Focal Loss -- A loss function that focuses on HARD examples and ignores easy ones. Like a student who studies the topics they keep failing instead of the ones they know.

Fold -- One split of the data in cross-validation. 5-fold CV = 5 different train/val splits.

G

GELU -- An activation function (Gaussian Error Linear Unit). Decides which information passes through. Smoother than ReLU.

GPU (Graphics Processing Unit) -- A processor optimised for parallel computation. Training deep learning models on GPUs is 10-100x faster than on CPUs.

Grad-CAM -- A technique to visualise which pixels most influenced a model's decision by computing gradients of the output with respect to the last convolutional layer.

Gradient -- The direction and magnitude of the steepest increase in the loss function. We go in the OPPOSITE direction to reduce the loss.

Gradient Accumulation -- Accumulating gradients over multiple batches before updating weights. Allows effective larger batch sizes on limited GPU memory.

Gradient Clipping -- Capping gradients at a maximum value. Prevents them from exploding (growing too large and destroying the model).

Gradient clipping -- Limiting the maximum gradient value to prevent "exploding" updates.

H

Head -- The final layers of a neural network that make the actual prediction. The "decision maker" after the backbone has extracted features.

Held-out set -- A portion of data never used during training or validation, reserved only for final evaluation.

Hyperparameter -- A setting you choose before training (like learning rate, batch size). The model does not learn these; you set them.

I

ImageNet -- A dataset of 14 million labelled images (dogs, cats, cars, etc.). Models pretrained on ImageNet have learned general visual features.

Imbalance -- When one class has many more examples than another. 82% good vs 18% defective = imbalanced.

Inference -- Using a trained model to make predictions on new data (test time).

Invariant -- A property that doesn't change under transformations. "Defect detection should be invariant to rotation" = a scratch is still a scratch when rotated.

K

K-Fold Cross-Validation -- Training K models, each on a different 80/20 split of the data. Gives a robust estimate of performance.

L

Label -- The correct answer for an image (0 = good, 1 = defective).

Label Smoothing -- Softening the labels from [0, 1] to [0.02, 0.98]. Prevents the model from being overconfident.

Label smoothing -- Replacing hard labels (0 or 1) with soft labels (0.02 or 0.98) to prevent overconfidence.

Learning rate -- How large each weight update step is. Too high = unstable training. Too low = very slow training.

Learning Rate (LR) -- How big each step is when the model updates its weights. Too big = overshoot the optimum. Too small = take forever.

Logit -- The raw output of a neural network before sigmoid. Can be any number (positive or negative). Sigmoid converts it to a probability (0 to 1).

Loss -- A number that measures how wrong the model is. Higher = more wrong. The model tries to make this number as small as possible.

Loss function -- A measure of how wrong the model's predictions are. Training minimises this number.

M

MixUp -- A data augmentation that blends two images and their labels linearly (e.g., 70% image_A + 30% image_B).

Model -- A mathematical function (neural network) that takes an image and outputs a prediction.

Momentum -- In optimizers, a smoothing factor that carries information from previous gradients to stabilise updates.

Multi-head attention -- Running several attention mechanisms in parallel, each focusing on different aspects of the input.

N

NaN (Not a Number) -- A numerical error that occurs when a computation produces an undefined result (like dividing by

zero). The training loop checks for and skips NaN losses.

Neural network -- A computational model inspired by the brain, made of layers of mathematical operations that transform input data into predictions.

Neuron -- A single computational unit in a neural network. Takes inputs, multiplies by weights, adds bias, applies activation.

Normalisation -- Scaling numbers to a standard range. Makes training more stable.

NumPy -- A Python library for working with arrays of numbers. The foundation of scientific computing in Python.

O

OHEM (Online Hard Example Mining) -- A training strategy that computes the loss per sample and only backpropagates through the hardest (highest-loss) examples in each batch.

Optimizer -- The algorithm that updates the model's weights to reduce the loss. AdamW is one type.

Overfitting -- When a model memorises training data instead of learning general patterns. It performs well on training data but poorly on new data.

Oversampling -- Duplicating examples from the minority class to balance the dataset.

P

Parameter -- A weight in the neural network. Learned during training. A small model has ~5 million parameters, a large model has ~50 million.

Parameters -- The learned values inside a neural network (weights and biases). EfficientNet-B0 has ~5M parameters; EnsembleV6 has ~50M+.

Phase -- A stage of training with different layers frozen/unfrozen. Phase 1 = head only, Phase 2 = head + last blocks, Phase 3 = everything.

Platt Scaling -- A calibration method that fits a logistic regression to make probabilities match reality.

Platt scaling -- A post-processing step that calibrates the model's output probabilities using logistic regression.

Precision -- Of all images predicted as defective, what fraction actually is defective? High precision = few false alarms.

Pretrained -- A model that was already trained on a large dataset (like ImageNet). We start from these weights instead of random.

Pseudo-Label -- A label guessed by a trained model for unlabelled data. Used to augment the training set.

Pseudo-labeling -- Using a trained model to predict labels for unlabelled data, then training on those predictions as if they were ground truth.

R

Recall -- Of all truly defective images, what fraction did the model catch? High recall = few missed defects.

Regularisation -- Techniques that prevent overfitting (dropout, weight decay, label smoothing, etc.).

ReLU -- An activation function. Outputs the input if positive, 0 if negative. Simple and effective.

Resume -- Continuing training from a saved checkpoint instead of starting from scratch.

RGB / BGR -- Red-Green-Blue vs. Blue-Green-Red. Different colour channel orders used by different libraries. OpenCV uses BGR, PIL uses RGB.

ROC AUC -- Area Under the Receiver Operating Characteristic curve. A threshold-independent measure of model quality.

S

Seed -- A number that initialises the random generator. Same seed = same "random" numbers = reproducible results.

Sigmoid -- A function that squashes any number into the range [0, 1]. Used to convert raw logits into probabilities.

Softmax -- Like sigmoid but for multiple classes. Converts scores to probabilities that sum to 1.0.

Stage 1 -- The fast EfficientNet-B0 filter in the cascade pipeline.

Stage 2 -- The powerful 5-fold ensemble in the cascade pipeline.

Stratified Split -- Splitting data while preserving the class ratio. If the full dataset is 18% defective, the validation set is also 18% defective.

SWA (Stochastic Weight Averaging) -- Averaging the model's weights over the last N epochs. Similar to EMA but averages EQUALLY (not exponentially).

Swin Transformer -- A Vision Transformer that uses shifted windows for efficient global attention.

T

Tensor -- A multi-dimensional array. A 3×256×256 tensor = 3 colour channels, 256 rows, 256 columns.

Test-Time Augmentation (TTA) -- Running inference multiple times with different transformations and averaging results.

Threshold -- The cutoff probability for classification. If threshold = 0.4 and probability = 0.45 -> defective.

torch.compile -- A PyTorch feature that compiles the model for faster execution. Like translating a book before reading it instead of translating each sentence.

Transfer Learning -- Starting from a pretrained model and fine-tuning it on your data. Much faster than training from scratch.

Transformer -- A neural network architecture that uses "attention" to process all parts of the input simultaneously. Originally for language, now used for images.

V

Validation -- Testing the model on data it hasn't seen during training. Measures real performance.

Vision Transformer (ViT) -- A Transformer applied to images by dividing them into patches (like words in a sentence).

W

Weight -- A learned parameter in a neural network that determines how much influence each input has.

Weight Decay -- A regularisation technique that penalises large weights. Prevents overfitting.

Worker -- A separate process that loads images in parallel while the GPU is training.

***End of Guide** -- You now know everything about the rim defect detection notebook (v4)!*

*New in this version: **OHEM** (Section 26) and **Grad-CAM Dynamic Cropping** (Section 27).*

If something is still unclear, check the specific cell in the notebook or look at the glossary again.
